

APPLYING REINFORCEMENT LEARNING METHODS TO PHYSICAL SYSTEMS

by

JACOB KRUSE

(Under the Direction of Prashant Doshi)

ABSTRACT

This section of the page is where your abstract will go. Fill in this spot in the dissertation.tex file with your abstract. Doctoral 350 words max, 150 max for masters.

INDEX WORDS: Model-Based Reinforcement Learning, Offline Reinforcement Learning, Robotic Grasping, Robotic Manipulators, Regularization, Ensemble Networks

APPLYING REINFORCEMENT LEARNING METHODS TO PHYSICAL SYSTEMS

by

JACOB KRUSE

Bachelor of Science, Western Illinois University, 2024

A Thesis Submitted to the Graduate Faculty of the
University of Georgia in Partial Fulfillment of the Requirements for the Degree.

MASTER OF SCIENCE

ATHENS, GEORGIA

2026

©2026

Jacob Kruse

All Rights Reserved

APPLYING REINFORCEMENT LEARNING METHODS TO PHYSICAL SYSTEMS

by

JACOB KRUSE

Major Professor: Prashant Doshi

Committee: Jin Lu

Mehmet Kurum

Electronic Version Approved:

Ron Walcott

Dean of the Graduate School

The University of Georgia

May 2026

DEDICATION

ACKNOWLEDGMENTS

CONTENTS

Dedication	iv
Acknowledgments	v
List of Figures	vii
List of Tables	x
1 Introduction	I
1.1 Contributions	3
1.2 Thesis Structure	3
2 Related Work	5
2.1 Learning Success Models for Grasping	5
2.2 Robust Reinforcement Learning	6
2.3 Offline Reinforcement Learning	7
2.4 Model-Based Reinforcement Learning	9
3 Preliminaries	II
3.1 Contextual Bandits	11
3.2 Policy Gradient Reinforcement Learning	12
3.3 Reward Function Modeling	16

3.4	Regularization in Offline Reinforcement Learning	20
4	Methodolgy	22
4.1	Approximating and Accurate Reward Model	22
4.1.1	Initial Data Collection Strategy	23
4.1.2	Leveraging Ensemble Networks	23
4.2	Iterative Policy Refinement	24
4.2.1	Maximum Action Noise Threshold Objective	25
4.2.2	Iterative Deployment of Trained Policy	26
5	Experiments	28
5.1	Experimental Setup	28
5.1.1	Task Description	28
5.1.2	Context Estimation	29
5.1.3	Context Definition	31
5.1.4	Action Definition	33
5.1.5	Reward Definition	35
5.1.6	Initial Dataset	36
6	Conclusion	38
6.1	Future Work	38
	Appendices	39
	A	39
	Bibliography	40

LIST OF FIGURES

2.1	Offline Reinforcement Learning. An exploration policy π_β interacts with the environment to collect a dataset of state-action transitions and rewards, which is stored in the buffer $\mathcal{D}$. This dataset is used to train a policy π , which is then deployed in the environment without further interaction.	8
2.2	Model-Based Reinforcement Learning. A policy π_k is executed in the real world to collect data, which populates the real world buffer $\mathcal{D}$. This data is used to learn an approximate transition function $\hat{f}_k$. The learned transition function is used to generate a simulated dataset $\hat{\mathcal{D}}$, which is combined with the real-world dataset $\mathcal{D}$ to update the policy π_{k+1} . This iterative process improves sample efficiency by utilizing simulated data while grounding learning in real world experience.	10
4.1	Iterative Reward Model and Policy Training. An exploration policy π_β and expert policy π_E interact with the environment to collect state-action-reward tuples, which are stored in the initial dataset $\mathcal{D}$. This dataset is used to train an approximate reward model $\tilde{r}_i$, which guides the optimization of a policy π_i . The learned policy π_i is deployed in the environment to generate on-policy data and adds it to the dataset $\mathcal{D}$. The reward model $\tilde{r}_{i+1}$ is updated with the augmented dataset, and a new policy π_{i+1} is optimized. This process repeats iteratively, progressively improving the reward model and policy.	27

5.1	Experimental Setup. A KUKA robotic manipulator is used to grab aluminum cans and plastic bottles located on a conveyor belt. An Oak-D RGB-D camera captures images of the scene, from which the 3D position and orientation of each item is calculated. With this information, the manipulator executes grasping actions to move the recyclable items to a recycling bin	29
5.2	Context Estimation. Figure 5.2a shows the original RGB image. This image is passed through a YOLO v11 instance segmentation model to obtain masks of the bottles and cans, as shown in Figure 5.2b. Principal Component Analysis (PCA) is performed on the masks to determine the centroid and principal axes, shown in Figure 5.2c	31
5.3	Discretized Workspace: Figure 5.3a shows the xy-plane of the workspace divided into four quadrants. In this example, the bottle is located in the second quadrant. Figure 5.3b illustrates the discretization of the possible yaw angles into four bins; the bottle falls within the first bin in this example. Similarly, the possible pitch angles are discretized into three bins, with the bottle located in the first bin, as shown in Figure 5.3c.	33
5.4	Local Coordinate Frame. The yellow dot on the bottle represents its centroid, while the red, green, and blue axes indicate the positive directions of the object's local x , y , and z axes, respectively. An action containing a $+\Delta l$ component would produce a translation along the red axis, starting from the yellow dot.	35

5.5 Expected Reward Comparisons: Each figure illustrates the expected reward $\mathcal{R}$ for a given state as the Δw component of the action is varied, comparing two policies trained on the same expected reward model. The blue lines correspond to the base action of the policy trained with the modified objective proposed in this work, while the orange lines show the same for the policy trained with the standard learning objective. The blue dot and orange dot indicate the respective base actions for each policy. Figure 5.5a directly compares the expected rewards of the base actions from both policies. Figures 5.5b and 5.5c show how the expected rewards change after applying the same adversarial perturbation to each action. Specifically, Figure 5.5b shows the effect of perturbing the action dimensions by 1 cm and 6° , while Figure 5.5c shows the effect of larger perturbations of 2 cm and 12° . 36

LIST OF TABLES

I.I	Hyper Parameters	4
-----	----------------------------	---

CHAPTER I

INTRODUCTION

Reinforcement Learning (RL) has achieved remarkable success in simulation, where agents can execute many actions and easily evaluate rewards. In simulated environments, large amounts of data can be generated quickly and safely, allowing policies to improve through extensive trial and error. For instance, (Vecerik et al., 2017) trains a robotic manipulator in simulation to perform a variety of manipulation tasks, while (Mnih et al., 2015) applies RL to teach an agent to play Atari games. These methods demonstrate the ability of RL algorithms to learn complex behaviors when there are large amounts of interaction data available to the agent.

Recently, RL has been increasingly applied to real-world tasks, motivated by the desire to tackle challenging problems beyond simulation. For example, (Smith et al., 2022) demonstrates how a legged robot can learn to traverse real world terrains, and (Wu et al., 2023) shows the application of their method to multiple physical platforms, including a quadruped robot, a robotic manipulator, and Sphero robot. Another prominent application of real-world RL is robotic grasping (Han et al., 2023), due to its widespread relevance in manipulation tasks such as warehouse automation and manufacturing. In these domains, robust grasping strategies are essential for practical deployment.

Despite this progress, the application of RL in the real world remains significantly more difficult than in simulation. The use of physical systems introduces uncertainty in sensing and actuation, while environmental factors introduce variability that further complicates learning. Additionally, robotic tasks

often involve high-dimensional continuous state and action spaces, leading to challenges associated with the curse of dimensionality. Data collection is also costly and time-consuming, as it requires executing actions on physical hardware. Furthermore, safety must be considered to avoid damaging equipment or causing injury. These challenges make it difficult to apply traditional RL methods, which typically rely on large amounts of accurate data obtained through online interaction.

Offline Reinforcement Learning is a popular strategy for addressing these limitations in physical systems. Offline RL methods learn policies entirely from pre-collected datasets without requiring additional environmental interaction during training. This greatly improves sample efficiency by enabling the algorithm to focus entirely on policy optimization rather than exploration. However, Offline RL introduces its own challenges, most notably distribution shift. Since the learned policy may select actions that differ from those present in the dataset, the value estimates for these actions cannot be evaluated reliably.

To mitigate this issue, many offline RL approaches incorporate techniques such as regularization and policy constraints. Regularization involves adding a term to the learning objective that encourages desirable behaviors in the learned policy, such as safer actions, improved exploration, or policy outputs that remain close to the collected data distribution. Incentivizing these behaviors can help stabilize training and reduce the risk of exploiting inaccuracies in value estimates.

Similarly, Maximum Entropy Reinforcement Learning (MaxEnt RL) (Haarnoja et al., 2017) alters the standard RL objective by incorporating an entropy term to the reward. This encourages the policy to maximize both expected return and entropy, promoting stochasticity in the policy and encouraging broader action distributions. While this resembles regularization with the additional term, MaxEnt RL fundamentally redefines the optimization objective. A well-known example is Soft Actor-Critic (SAC) (Haarnoja et al., 2018), which optimizes the MaxEnt objective and has demonstrated strong performance in continuous control tasks.

Inspired by these approaches, this work proposes an alternative objective that promotes robustness in the selected actions. A policy that chooses actions that perform well in ideal conditions but fail in practice due to noise from imperfect actuation, sensing, or environmental disturbances is undesirable. Therefore,

we encourage the policy to select actions that not only achieve high reward but also remain effective under small perturbations.

I.1 Contributions

In this paper, we introduce a framework that learns an approximate reward model and leverages it for iterative policy optimization with a novel robustness-oriented objective. Our approach is inspired by ideas from Soft Actor-Critic and Maximum Entropy Reinforcement Learning. The data collection strategy helps provide a diverse dataset for learning an accurate initial reward model. By learning an approximate reward function, we are able to efficiently and accurately predict rewards for a large number of state-action pairs. Focusing on action robustness during optimization encourages policies that are more tolerant to perturbations in real-world execution. Finally iterative deployment allows the method to obtain on-policy data and evaluate the effectiveness of the converged policy before further improvement. The main contributions of this work are:

- A new learning objective, the **I Don’t Know Yet (IDKY)**, which encourages selection of actions that are both high-reward and robust to perturbations.
- Experiments demonstrating the method on a robotic sorting task, showing improved performance and robustness compared to standard policy optimization.
- An iterative learning algorithm that refines both the reward model and policy by combining expert, random, and on-policy data to efficiently improve grasping performance.

I.2 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2 - Related Work:** This chapter reviews prior work on Learning Grasp Success Models, Robust Reinforcement Learning, Offline Reinforcement Learning, and Model-Based Reinforce-

ment Learning. While these areas share similarities with the proposed method, they differ from the approach presented in this work.

- **Chapter 3 - Preliminaries:** This chapter introduces the foundational concepts necessary to understand the proposed method. Section 3.1 defines the *blank* framework used to model the grasping problem, while Section 3.2 describes how reward models can be learned using supervised learning techniques.
- **Chapter 4 - Methodology:** This chapter presents the proposed method in detail. Section 4.1 describes strategies used to learn an accurate reward model, including an effective data collection strategy and the use of ensemble networks capture uncertainty. Section 4.2 introduces the main algorithm for policy learning, which utilizes the newly defined IDKY objective and an iterative deployment strategy to progressively improve the the policy.
- **Chapter 5 - Experiments:** This chapter describes the experimental evaluation of the method. It includes details of the robotic grasping setup, datasets used, evaluation metrics, and results demonstrating the robustness of the proposed approach.
- **Chapter 6 - Conclusion:** This final chapter summarizes the contributions of the thesis, discusses the implications of the results, and outlines possible directions for future work.

Table 1.1: Hyper Parameters

Hyper Parameter	Value
α	1.25
β	0.75
δ_t	[0.025, 0.025, 0.025, 0.125, 0.125, 0.125]

CHAPTER 2

RELATED WORK

2.1 Learning Success Models for Grasping

Robotic grasping is a major area of study within robotics due to its wide use in manipulation tasks. As these tasks become more complex, traditional hand-engineered approaches are becoming increasingly ineffective. As a result, a substantial portion of research in robotic grasping has shifted toward learning-based methods that leverage datasets to model grasp outcomes. Many of these approaches focus on learning a grasp success model that predicts whether a proposed grasp action will succeed in the current state of the environment.

These models are trained to predict the success probability of candidate grasps using information such as RGB images, depth maps, object positions, or affordance representations. For example, after collecting data from 50,000 grasp attempts, (Pinto & Gupta, 2016) trained a Convolutional Neural Network (CNN) that takes image patches as input and outputs the probability of grip success for a set of candidate grasp angles. Similarly, (Levine et al., 2018) learned a grasp prediction network that predicts the likelihood of a successful grasp from visual input and integrated this estimate within a continuous visual servoing controller.

Other work has explored the use of synthetic datasets to train these grasp evaluation models. After generating a large dataset of synthetic point clouds, (Mahler et al., 2017) learned a grip robustness function

that evaluates candidate grasps and enables planning by selecting the highest-scoring action. This approach demonstrates how simulated data can be used to train these models while allowing them to generalize to real world tasks. Other methods, such as (Asif et al., 2019) and (Detry et al., 2011), learn these grasp success models from interaction data and utilize them for simple evaluation or to find high success actions.

The method in this paper follows this line of work by learning a grasp success model that maps states and actions to success probabilities. However, rather than using this learned model solely as an evaluation metric for grasp selection, the success predictor is interpreted as a reward function in this work. This reward model is then incorporated into a reinforcement learning framework to optimize an effective grasping policy.

2.2 Robust Reinforcement Learning

Robust Reinforcement Learning (Moos et al., 2022) encompasses a class of methods that extend the standard reinforcement learning objective to account for hostile environmental effects or adversarial factors. These approaches are especially useful for real world applications, where uncertainty and disturbances are inherent to the environment. The goal therefore is to learn a policy that achieves high expected reward while remaining effective in the presence of noise, adversaries, and uncertainty.

Prior work in Robust RL can be categorized based on the source of uncertainty that the method aims to address. The most common approaches focus on robustness with respect to uncertainty in the transition dynamics or reward function. For example, (Bagnell et al., 2001) accounts for the stochastic nature of the environment by reformulating the reinforcement learning objective as a minimax problem, where the policy is optimized against the worst-case dynamics drawn from the set of possible transition probabilities. Additionally, (Xu & Mannor, 2006) learns a policy that maximizes a combination of a performance and robustness measure, where the robustness component captures the worst-case reward for the evaluated action. By optimizing against the worst-case outcomes in the reward or transition dynamics, these approaches produce policies that are robust but often conservative, sometimes resulting in reduced performance.

Many formulations that address transition and reward uncertainty are limited to tabular settings and do not easily extend to nonlinear function approximators or continuous state-action spaces. Another line of research in Robust RL instead focuses on robustness to adversarial or noisy actions, which can be applied in these more complex settings. The work of (Tessler et al., 2019) introduces the Noisy Action Robust MDP, where the standard policy is replaced with a joint policy π_{mix} that perturbs the intended action through an adversarial policy. The resulting optimization is formulated in a minimax framework for the mixed policy. Similarly, (Tan et al., 2020) defines a robust optimization objective in which adversarial perturbations are applied to actions in order to minimize the expected reward before policy optimization occurs. Much like the transition and reward methods, these methods aim to learn policies that perform well under worst-case action perturbations.

The objective proposed in this work draws a significant amount of inspiration from Robust RL techniques. However, rather than optimizing for the worst-case scenario like the methods described above, our approach instead optimizes with respect to a fixed perturbation magnitude to the actions. This formulation aims to produce policies that remain effective under environmental uncertainty without becoming overly conservative, thereby balancing robustness and performance.

2.3 Offline Reinforcement Learning

Offline Reinforcement Learning (Prudencio et al., 2023) focuses on learning a policy π entirely from a fixed dataset $\mathcal{D}$ that has been previously collected by an exploration or behavior policy π_β , without any additional interaction with the environment (see Figure 2.1). This approach contrasts with Online Reinforcement Learning, where the agent continually collects new experience while learning. Despite this distinction, the main objective of Offline RL is the same as standard Online RL, which is to learn a policy that maximizes expected cumulative reward. Because training is performed on static, batched data, Offline RL is often more sample-efficient and can converge faster than Online RL, where interactions with the environment may be costly and slow down the learning process.

However, a central challenge with Offline RL is distribution shift. During training, the learned policy π may select actions or reach states that were rarely or never encountered in the dataset $\mathcal{D}$. This mismatch between the training data distribution and the states and actions encountered by the learned policy can result in unreliable value estimation, which in turn may lead to poor policy performance when deployed. To address this issue, prior work has developed a range of strategies. Techniques such as importance sampling (Precup et al., 2000) help correct the bias induced by off-policy evaluation, while regularization methods (Kumar et al., 2020) can encourage desirable behaviors in the learned policy that may not be sufficiently represented in the dataset $\mathcal{D}$. Similarly, policy constraints (Fujimoto et al., 2019) can be used to keep the learned policy π close to the dataset $\mathcal{D}$, improving the reliability of value estimation.

Although the method presented in this paper incorporates principles from Offline RL, it does not implement a pure offline approach. The learned policy π is subsequently deployed in the environment to acquire more data, which is then used to iteratively refine the reward model for further offline training. Despite this, the strategy of regularization formed a primary inspiration for this work. The combination of offline training with online data collection gives this method the efficiency of Offline RL while also addressing the limitations of static datasets.

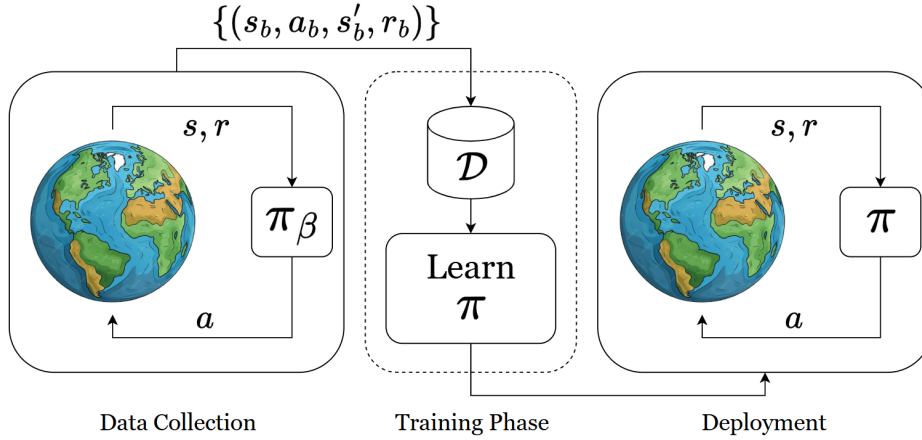


Figure 2.1: Offline Reinforcement Learning. An exploration policy π_β interacts with the environment to collect a dataset of state-action transitions and rewards, which is stored in the buffer $\mathcal{D}$. This dataset is used to train a policy π , which is then deployed in the environment without further interaction.

2.4 Model-Based Reinforcement Learning

Model-Based Reinforcement Learning (Luo et al., 2024) is a class of methods that learn a model of the environment’s dynamics or transition function, which predicts the resulting state of actions taken by the agent. The learned transition model allows the agent to simulate future states and generate synthetic data without interacting with the real environment. This is particularly useful in domains where real world interactions are costly, dangerous, or time consuming. Leveraging a learned model, these methods significantly improve sample efficiency since the agent can perform planning and optimization on a combination of real and simulated experiences. A basic Model-Based RL pipeline is illustrated in Figure 2.2.

A key challenge in Model-Based RL is model accuracy and uncertainty. Several approaches address this issue through uncertainty-aware modeling, such as ensemble methods which train multiple independent models. One approach, proposed in (Matsushima et al., 2020), trains an ensemble of K dynamics models and randomly selects one for each rollout, which helps reduce overfitting and increases robustness to individual errors. Additionally, (Yu et al., 2020) learns N reward models and estimates the reward as the ensemble mean penalized by the maximum standard deviation to incentivize the policy to avoid regions of high uncertainty.

In addition to ensemble methods, many Model-Based RL methods adopt iterative model refinement and Dyna-style (Sutton, 1991) techniques to continuously improve the dynamics model and policy. Works such as (Feinberg et al., 2018) and (Nagabandi et al., 2018) follow this paradigm by alternating between model learning and data collection with the optimized policy to progressively improve performance. This leads to more robust and effective agents, especially in complex environments.

Although the proposed approach is related to Model-Based RL, it does not explicitly model a transition function. Instead, the method focuses on learning a reward model, which estimates the expected return of actions directly. Using the reward predictions, the policy can be optimized without the need for modeling state transitions. Despite this distinction, the approach adopts ideas from Model-Based RL, including

the use of ensemble networks to capture uncertainty and iterative model improvement to enhance model accuracy.

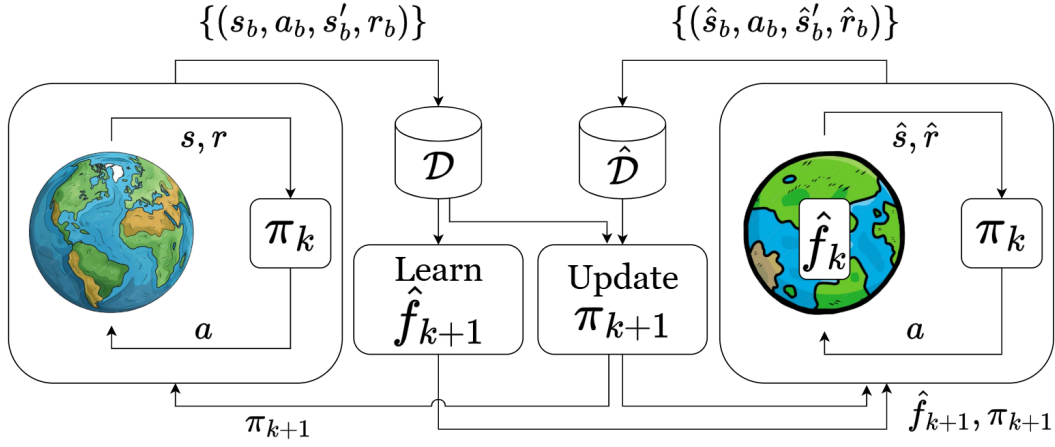


Figure 2.2: Model-Based Reinforcement Learning. A policy π_k is executed in the real world to collect data, which populates the real world buffer $\mathcal{D}$. This data is used to learn an approximate transition function $\hat{f}_k$. The learned transition function is used to generate a simulated dataset $\hat{\mathcal{D}}$, which is combined with the real-world dataset $\mathcal{D}$ to update the policy π_{k+1} . This iterative process improves sample efficiency by utilizing simulated data while grounding learning in real world experience.

CHAPTER 3

PRELIMINARIES

3.1 Contextual Bandits

The Multi-Armed Bandit (Slivkins, 2024) framework is one of the most widely used models for decision making under uncertainty. In this setting, a single agent chooses from K arms or actions in each round and receives a reward from an unknown distribution associated with the chosen arm. After observing the reward, the agent repeats this process by selecting from the same set of arms, making it a sequential problem with inherent uncertainty. The objective is to both learn the reward distribution of each arm and to maximize the expected reward.

A common extension of the Multi-Armed Bandit model is the Contextual Bandit framework (Langford & Zhang, 2007). Similar to the previous formulation, the agent chooses from a set of K arms or actions; however, in this case, the reward distribution now depends on an observed context. Despite this added structure, the goal of the Contextual Bandit problem remains the same as the Multi-Armed Bandit, to maximize reward in a stochastic and unknown environment. The Contextual Bandit (CB) can be formally defined with the tuple

$$\text{CB} = \langle \mathcal{X}, \mathcal{A}, R \rangle, \tag{3.1}$$

where $\mathcal{X}$ denotes the context space, $\mathcal{A}$ represents the action space, and R is the reward function. These components are explained below:

- **Context Space:** The context space $\mathcal{X}$ is the set of all possible contexts in the environment. In each round, a context $x \in \mathcal{X}$ is observed, representing the current state of the environment. The context x is drawn from an underlying context distribution ρ_x . The context may be discrete, continuous, or multi-dimensional, and is assumed to be fully observable.
- **Action Space:** The action space $\mathcal{A}$ is the set of all possible actions available to the agent. Given a context x , the agent selects an action $a \in \mathcal{A}$ and executes it in the environment. The action may also be discrete, continuous, or multi-dimensional.
- **Reward Function:** The reward function R is used to determine the reward r after each round. The reward function R is a function of the context x and action a and returns a distribution of real-valued rewards $R : \mathcal{X} \times \mathcal{A} \rightarrow \Delta_{\mathbb{R}}$. The observed reward r is then sampled from the reward function $r \sim R(x, a)$ during each round. The underlying reward distribution induces an expected reward function $\mathcal{R}(x, a) = \mathbb{E}_{r \sim R(x, a)}[r]$, which represents the average reward for each state-action pair.

In order to determine which actions to take in each context, the agent implements a policy π , which defines a mapping from the context space to the action space. A policy can either be stochastic or deterministic, depending on the desired behavior. A stochastic policy maps a given context x to a probability distribution over the actions $\pi : \mathcal{X} \rightarrow \Delta_{\mathcal{A}}$. To select an action, the agent samples the action a from the resulting distribution $a \sim \pi(\cdot|x)$. In the deterministic case, the policy maps each context to a single action $\pi : \mathcal{X} \rightarrow \mathcal{A}$, such that $a = \pi(x)$ with a probability of 1.

3.2 Policy Gradient Reinforcement Learning

Reinforcement Learning (RL) (Sutton, Barto, et al., 1998) is one of the most extensively studied paradigms in artificial intelligence. It can be described as the process of an agent learning rational, favorable behavior

through interactions with its environment. A wide variety of techniques fall under the umbrella of RL, but they are generally categorized into two broad classes: Model-Based and Model-Free methods. In Model-Based RL (see Section 2.4), the agent tries to learn a model of the environment and uses this model for planning or predicting outcomes. In contrast, Model-Free RL bypasses the need for a model and aims to learn through direct interaction with the environment. Model-Free methods are highly popular due to their simplicity, robustness, and ability to scale to complex domains.

To take this a step further, Model-Free RL methods can be divided even further into Policy Gradient, Value-Based, and Actor-Critic techniques. In Policy Gradient approaches, the agent directly optimizes its policy by interacting with the environment and using the observed rewards to guide policy updates. These approaches are well-suited for continuous action spaces, as they focus solely on the policy, but are generally restricted to being on-policy due to the nature of policy optimization. In contrast, Value-Based RL methods focus on learning a function that estimates the value of states or actions, which is then used to guide action selection. These methods are typically more sample-efficient than Policy Gradient techniques because they can reuse past data, but they can struggle with large or continuous state and action spaces. Finally, Actor-Critic methods combine the strengths of Policy Gradient and Value-Based approaches. In this framework, the actor represents the policy and the critic refers to the value function. The feedback of the critic is used to update the actor, leading to more stable and efficient learning compared to the either method alone.

Despite the wide variety of RL algorithms, the underlying learning objective remains the same: to find a policy π that maximizes the expected reward. The most general form of the learning objective is defined with the Markov Decision Process (MDP) framework (Bellman, 1957), and is commonly expressed as

$$J(\pi) = \mathbb{E}_{s_0 \sim \rho_s, a_t \sim \pi(\cdot|s_t), s_{t+1} \sim \mathcal{T}(s_t, a_t)} \left[\sum_{t=1}^T R(s_t, a_t) \right], \quad (3.2)$$

where the initial state s_0 is sampled from the state distribution ρ_s , actions a_t are sampled from the stochastic policy π , resulting states s_{t+1} are sampled from the transition function $\mathcal{T}(s, a)$, and rewards are given by the reward function $R(s, a)$ which is assumed to be deterministic.

To adapt this learning objective to the Contextual Bandit setting, some changes need to be made. Since Contextual Bandits involve no state/context transitions and no notion of sequential decision-making, the transition function $\mathcal{T}(s, a)$ and the summation over all time steps in an episode can be removed. Additionally, the reward function is typically modeled as stochastic, with observed rewards sampled as $r \sim R(x, a)$. This can be handled either by taking an expectation over sampled rewards or, equivalently, by directly using the expected reward function $\mathcal{R}(x, a)$, which simplifies the formulation while remaining consistent with the original objective.

On top of all this, a deterministic policy is implemented, where each context x is mapped to a fixed action $a = \pi(x)$. Under these modifications, the learning objective becomes

$$J(\pi) = \mathbb{E}_{x \sim \rho_x} [\mathcal{R}(x, \pi(x))], \quad (3.3)$$

where contexts x are sampled from the context distribution ρ_x , rewards are given by the expected reward function $\mathcal{R}(x, a)$, and actions are deterministically selected by the policy π .

Although Contextual Bandits may employ either stochastic or deterministic policies, this work focuses on deterministic policies; therefore, Equation 3.3 is adopted as the learning objective. Under this formulation, the optimal policy π^* is defined as the policy that maximizes expected reward:

$$\pi^* = \arg \max_{\pi} J(\pi). \quad (3.4)$$

With the advancement of Deep Reinforcement Learning (Y. Li, 2017), it has become increasingly common to represent policies using neural networks. A neural network (Gurney, 2018) can be described as a parameterized function composed of multiple layers and interconnected nodes, where each layer applies weights, biases, and activation functions. After receiving an input, the network propagates it through its layers to produce an output. In the RL setting, the neural network acts as the policy, taking a state or context representation as input and producing an action or distribution over actions as output. The use of these networks enables highly expressive, nonlinear function approximation, allowing policies

and value functions to handle high-dimensional and complex domains. When a policy is parameterized by a neural network with parameters ϕ , it is denoted as π_ϕ . The learning objective can then be written as a function of these parameters:

$$J(\phi) = \mathbb{E}_{x \sim \rho_x} [\mathcal{R}(x, \pi_\phi(x))]. \quad (3.5)$$

Importantly, the objective itself has not changed, but instead, it is now expressed in terms of the parameters ϕ that represent the policy. This shift is crucial, as it transforms the reinforcement learning problem into an optimization problem over the parameter space.

This change gives rise to the central idea behind Policy Gradient methods, which is to optimize the parameters ϕ using the gradient of the objective. More specifically, the goal is to iteratively update the parameters ϕ in the direction of the gradient $\nabla_\phi J(\phi)$, which quantifies how changes in the parameters affect the expected reward. For the Contextual Bandit setting, the deterministic policy gradient can be expressed as a modification of the original formulation in (Silver et al., 2014):

$$\nabla_\phi J(\phi) = \mathbb{E}_{x \sim \rho_x} [\nabla_\phi \pi_\phi(x) \cdot \nabla_a \mathcal{R}(x, a)|_{a=\pi_\phi(x)}]. \quad (3.6)$$

This expression differs from the original deterministic policy gradient formulation because it is adapted to the Contextual Bandit setting rather than the MDP framework (Bellman, 1957). In a standard MDP, the policy gradient depends on the Q-function $Q(s, a)$, which represents the total discounted sum of future rewards starting from state s and taking action a . The Q-function captures transition dynamics and is required to propagate future rewards back to earlier actions. In contrast, Contextual Bandits have no state or context transitions, so the policy does not need to account for future outcomes. As a result, the Q-function can be replaced by the immediate expected reward function $\mathcal{R}(x, a)$, allowing the deterministic policy gradient to be applied directly to Contextual Bandits.

Policy Gradient methods use the gradient of the learning objective to directly optimize the policy parameters. More specifically, the algorithm performs gradient ascent, adjusting the policy parameters

in the direction that increases the expected reward. At each update step, the parameters are adjusted according to

$$\phi_{k+1} = \phi_k + \lambda \cdot \nabla_{\phi} J(\phi_k), \quad (3.7)$$

where λ is the learning rate. This update effectively pushes the policy toward actions that yield higher rewards, reinforcing behaviors that are more likely to succeed.

In practice, the gradient of the learning objective cannot be calculated exactly because it depends on the full distribution of contexts and rewards, which is generally unknown. Instead, it must be estimated from sampled interactions with the environment. This can be done by executing the current policy π_{ϕ} in the real-world and adding the observed context, action, and reward tuples into a dataset $\mathcal{D}^{\pi_{\phi}} = \{(x_b, a_b, r_b)\}_{b=1}^{\|\mathcal{D}^{\pi_{\phi}}\|}$. This dataset provides a finite sample of the policy's behavior and its effects on the environment, which can be used to estimate the gradient. The deterministic policy gradient can then be approximated as

$$\nabla_{\phi} J(\phi) \approx \frac{1}{\|\mathcal{D}^{\pi_{\phi}}\|} \sum_{b=1}^{\|\mathcal{D}^{\pi_{\phi}}\|} \nabla_{\phi} \pi_{\phi}(x_b) \cdot \nabla_a \mathcal{R}(x_b, a_b)|_{a_b=\pi_{\phi}(x_b)}. \quad (3.8)$$

This expression is an unbiased Monte Carlo estimate of the true gradient. Although this estimate does not yield the exact gradient, it is sufficiently accurate to guide parameter updates in Policy Gradient methods, allowing the agent to iteratively improve its policy.

3.3 Reward Function Modeling

The reward function $R(x, a)$ for a Contextual Bandit, defined in Section 3.1, is stochastic and returns a distribution over real-valued rewards. After taking action a in context x , the observed reward r is sampled from this distribution $r \sim R(x, a)$. The expected reward function $\mathcal{R}(x, a)$ can be used equivalently to represent the sampled rewards, but it instead returns a scalar value. Because of this, if the expected

reward function is known, the policy can use it directly for optimization by calculating the gradient with Equations 3.6 or 3.8.

In many practical settings, however, the true reward function is unknown and must be inferred through interactions with environment. One approach is to learn a policy directly from past experience and utilize importance sampling (Precup et al., 2000). Since this experience is collected under an exploration policy π_β , whose action probabilities differ from the policy π_k being optimized, the expected reward cannot be accurately estimated without adjustment. Importance sampling addresses this by weighting observed actions according to how likely the trained policy π_k would select them, thereby producing a more accurate estimate of the expected reward under the target policy π_k .

Although importance sampling is powerful, it struggles with deterministic policies and continuous action spaces. As an alternative, one can use the collected data to learn a model that approximates the expected reward function $\hat{\mathcal{R}}(x, a)$, and use this in place of the true expected reward function $\mathcal{R}(x, a)$ for policy optimization. Under the exploration policy π_β , the agent observes tuples of the form (x_b, a_b, r_b) where x_b is the context, $a_b \sim \pi_\beta(\cdot | x_b)$ is the action taken, and r_b is the observed reward. These interactions can be used to generate a dataset $\mathcal{D} = \{(x_b, a_b, r_b)\}_{b=1}^{\|\mathcal{D}\|}$. Modeling the approximate reward function then becomes a Supervised Learning problem.

In Supervised Learning (Cunningham et al., 2008), the goal is to obtain a model that predicts the true labels of all input-output pairs in a dataset. To measure the discrepancy between the true labels y corresponding to input x and predicted labels $\hat{y}$ from the approximate model $\hat{f}(x)$, a loss function $\mathcal{L}(y, \hat{f}(x))$ is introduced (C. Li et al., 2025). The empirical risk $\mathcal{L}(\mathcal{D}, \hat{f})$ or the average loss of the model over an entire dataset is then defined as

$$\mathcal{L}(\mathcal{D}, \hat{f}) = \frac{1}{\|\mathcal{D}\|} \sum_{b=1}^{\|\mathcal{D}\|} \mathcal{L}(y_b, \hat{f}(x_b)). \quad (3.9)$$

This quantity measures the performance of the model on the observed data. The model that best fits the data $\hat{f}^*$ is obtained by minimizing the empirical risk:

$$\hat{f}^* = \arg \min_{\hat{f}} \mathcal{L}(\mathcal{D}, \hat{f}). \quad (3.10)$$

In practice, it is very common to represent the model using a neural network. For a model $\hat{f}_\theta$ parameterized by θ , the loss function can be rewritten as $\mathcal{L}(y, \theta)$, so that it is now a function of the true labels y and the parameters θ that represent the approximate model. With this change, Equation 3.10 is reframed as finding the parameters that minimize the empirical risk. This optimization problem is typically solved with gradient descent, where the parameters are iteratively updated in the direction of the negative gradient of the empirical risk. The parameters θ are then updated according to the following rule:

$$\theta_{k+1} = \theta_k - \lambda \cdot \nabla_{\theta} \mathcal{L}(\theta_k), \quad (3.11)$$

where λ is the learning rate.

In the setting of learning a model to approximate the expected reward function, the goal is to predict the true expected reward for each context-action pair. This means the reward model is trained by minimizing the loss between the true observed rewards and predicted expected rewards: $\mathcal{L}(r_b, \hat{\mathcal{R}}_\theta(x_b, a_b))$. Because rewards are obtained directly from interactions with the environment, no manual annotation is required. In this case, reward modeling can be viewed as a form of Self-Supervised Learning (Jaiswal et al., 2020), an effective branch of Supervised Learning that avoids the cost of labeling large datasets. For a self-annotated dataset $\mathcal{D}$, the optimal parameters θ are obtained by iteratively minimizing the empirical risk over sampled mini-batches of size B :

$$\theta^* = \arg \min_{\theta} \frac{1}{B} \sum_{b=1}^B \mathcal{L}(r_b, \hat{\mathcal{R}}_\theta(x_b, a_b)). \quad (3.12)$$

Once trained, this model can be used for policy optimization, enabling sample-efficient learning without access to the true expected reward function.

When applying Supervised Learning, it is important to choose a loss function that is appropriate for the given problem. In the context of learning an approximate expected reward function $\hat{\mathcal{R}}(x, a)$,

the model is trying to capture the true underlying stochastic reward function $R(x, a)$, which returns a probability distribution over discrete reward outcomes. Because of this, Cross-Entropy (CE) loss (Mao et al., 2023) is suitable, as it is widely used for probability models and classification tasks. The most general form of CE loss is Categorical Cross-Entropy loss $\mathcal{L}_{CCE}$, which is defined as

$$\mathcal{L}_{CCE}(\theta) = - \sum_{b=1}^C y_b \cdot \log(\hat{y}_b), \quad (3.13)$$

where C is the number of categories for the labels. The equation above is for a single-instance loss, so the corresponding empirical risk $\mathcal{L}_{CCE}$ for a mini-batch of size B is

$$\mathcal{L}_{CCE}(\theta) = - \frac{1}{B} \sum_{b=1}^B \sum_{c=1}^C y_{b,c} \cdot \log(\hat{y}_{b,c}). \quad (3.14)$$

Minimizing this objective encourages the reward model to assign high probability to observed rewards and low probability to others, effectively learning a distribution over rewards.

In many practical settings, the reward is binary, taking one of two labels (success or failure). In this case, Binary Cross-Entropy (BCE) loss is appropriate. The per-sample BCE loss $\mathcal{L}_{BCE}$ is

$$\mathcal{L}_{BCE}(\theta) = -y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y}), \quad (3.15)$$

and the corresponding empirical risk $\mathcal{L}_{BCE}$ over a mini-batch of size B is

$$\mathcal{L}_{BCE}(\theta) = - \frac{1}{B} \sum_{b=1}^B y_b \cdot \log(\hat{y}_b) + (1 - y_b) \cdot \log(1 - \hat{y}_b). \quad (3.16)$$

After training a reward model with BCE loss, the output represents the probability of receiving each of the two possible reward outcomes. For a binary reward system with values of 0 or 1, the expected reward simplifies to the probability of receiving 1:

$$\mathbb{E}_{r \sim R(x,a)}[r] = 0 \cdot Pr(0|x, a) + 1 \cdot Pr(1|x, a) = Pr(1|x, a)$$

Since a reward of 1 is generally associated with a successful action, the reward model can be interpreted as outputting the probability of success.

3.4 Regularization in Offline Reinforcement Learning

Although Offline Reinforcement Learning is effective, it is not without limitations. A key challenge arises from the fact that optimization is performed on a static dataset. This can lead to overfitting, where the learned policy performs well on the observed data but fails to generalize to unseen situations when deployed in the real world. These failures arise because the policy exploits biases in the dataset, which may be incomplete or does not sufficiently cover the relevant state and action spaces. This is particularly problematic because Offline RL does not have the ability to correct these errors with environmental feedback like standard Online RL algorithms.

To address these limitations, a common approach is to incorporate regularization into the learning process. In Offline RL, regularization refers to the addition of an extra term or function to the standard RL objective. This term is designed to penalize or encourage specific states, actions, or characteristics, effectively shaping the learned policy towards behaviors that are not fully represented in the dataset. The main idea is that the dataset is limited or biased, and the regularization term serves as a corrective mechanism, guiding the policy towards more favorable outcomes. Formally, for a policy π_ϕ parameterized by ϕ and a regularization function $\mathfrak{R}(\cdot)$, the regularized objective can then be written as

$$J(\phi) = \mathbb{E}_{x \sim p_x(\cdot)} [\mathcal{R}(x, \pi_\phi(x))] + \beta \mathfrak{R}(\cdot), \quad (3.17)$$

where the regularization term $\mathfrak{R}(\cdot)$ can depend on any relevant variables, including the context x , action a , or even the policy π_ϕ itself.

The regularization function $\mathfrak{R}(\cdot)$ is highly versatile and can be applied in multiple ways. For instance, adding a positive entropy term to the objective promotes stochasticity and encourages exploration in the policy, which is exemplified in (Haarnoja et al., 2018). Alternatively, regularization can be used to constrain

the learned policy to remain close to the dataset distribution. An example from (Nachum et al., 2019) is the use of a negative f -divergence term, which penalizes deviations from the dataset and ensures the policy remains supported by the data. Overall, regularization is a powerful tool that can induce desirable behaviors and compensate for the limitations of Offline RL, all while maintaining computational and sample efficiency.

CHAPTER 4

METHODOLOGY

This section details the paper’s approach to learn an effective policy. The method is organized into two distinct parts: Approximation of an Accurate Reward Model 4.1 and Iterative Policy Refinement 4.2.

4.1 Approximating and Accurate Reward Model

Before any policy learning takes place, a reward model must first be obtained. As detailed in Section 3.3, learning a reward model consists of utilizing a dataset $\mathcal{D}$ defined as $\mathcal{D} = \{(x_b, a_b, r_b)\}_{b=1}^{|\mathcal{D}|}$. The reward serves as the output label, and supervised learning is performed on the state-action inputs. Although this basic process can produce effective results, additional strategies are employed to further improve the accuracy and robustness of the approximate reward model. Implementing an effective data collection strategy and utilizing ensemble networks are the two primary strategies employed in this work. Together, these components help the learned reward model to produce more accurate and reliable reward estimates for state-action pairs. This accuracy is crucial, as the reward model is subsequently used to guide and optimize the policy.

4.1.1 Initial Data Collection Strategy

It is widely accepted that a model is only as good as the data on which it is trained. Ideally, a model is trained on a large and diverse dataset, enabling it to generalize across a wide range of states and actions. However, producing large-scale datasets is often time-consuming and expensive; therefore, it is essential to maximize diversity when working with limited data. To achieve this, the approach creates an initial dataset $\mathcal{D}_0$ with data generated by a human expert π_{Expert} and a random exploration policy π_{Random} . The term “expert” is used loosely here; while the actions should produce high rewards, they need not be strictly optimal. The purpose of the expert is to supply state-action pairs with relatively high rewards in order to provide a positive learning signal for the optimization procedure. In contrast, the random policy π_{Random} is likely to produce lower-reward actions, but its primary purpose is to provide a broad coverage of the state and action spaces. By combining these two data sources, the constructed dataset achieves both reward diversity and state-action coverage, improving the quality of the learned reward function.

4.1.2 Leveraging Ensemble Networks

A common strategy in supervised learning is to train an ensemble of networks rather than relying on a single model. This allows multiple predictions to be produced and aggregated in ways that improve robustness and provide estimates of uncertainty. In this work, N approximate reward models $\hat{r}_{\theta_1}, \dots, \hat{r}_{\theta_N}$ are trained independently with different initial parameters θ_i . The ensemble outputs are used to compute the mean predicted reward μ_θ and the standard deviation σ_θ , defined as

$$\mu_\theta(x, a) = \frac{1}{N} \sum_{i=1}^N \hat{r}_{\theta_i}(x, a), \quad (4.1)$$

$$\sigma_\theta(x, a) = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{r}_{\theta_i}(x, a) - \hat{\mu}_\theta(x, a))^2}. \quad (4.2)$$

The predicted mean reward μ_θ serves as the primary estimate of the approximated reward. The standard deviation σ_θ is used to quantify model uncertainty. Since independently trained models are likely to produce different predictions for state-action pairs that are poorly represented in the dataset, the standard deviation σ_θ provides a natural measure of uncertainty. Using these quantities, the uncertainty-penalized reward $\tilde{r}_\theta$ is defined as

$$\tilde{r}_\theta(x, a) = \mu_\theta(x, a) - \beta \cdot \sigma_\theta(x, a), \quad (4.3)$$

where the parameter β controls the degree to which uncertainty is penalized. Larger values of β will lead to more conservative reward approximations and will discourage the policy from selecting actions where the reward model is uncertain.

Algorithm 1 Train Reward Ensemble

Inputs: Dataset D , Number of reward networks N

Output: Reward model $\tilde{r}_\theta$

- 1: Initialize reward networks $\hat{r}_{\theta_1}, \dots, \hat{r}_{\theta_N}$ with random parameters $\theta_1, \dots, \theta_N$
 - 2: **for** $t = 1$ to T **do**
 - 3: For each network $\hat{r}_{\theta_n}$, sample an independent mini-batch B_n from dataset $\mathcal{D}$
 - 4: Update $\theta_n \leftarrow \arg \min_{\theta_n} \frac{1}{B_n} \sum_{b=1}^{B_n} \mathcal{L}(r_b, \hat{r}_{\theta_n}(x_b, a_b))$
 - 5: **end for**
 - 6: **return** $\tilde{r}_\theta$
-

4.2 Iterative Policy Refinement

Although it is common to compute a policy by directly maximizing the reward model over a set of candidate actions, this approach becomes significantly more challenging in continuous action spaces. As a result, policy optimization is adopted as the primary strategy. After training the approximate reward model, policy optimization can be carried out using the policy gradient defined in Equation 3.6, which has been shown to be an effective approach for learning policies. However, this work employs a modified learning objective together with an iterative policy learning process to achieve improved policy performance and greater robustness compared to standard optimization.

4.2.1 Maximum Action Noise Threshold Objective

The main contribution of this paper is a modified learning objective. In reinforcement learning, the main objective is to maximize expected reward, formalized in Equation 3.3. However, when optimizing this objective in the offline setting with an approximate reward model, two issues can arise. First, the policy may exploit inaccuracies in the reward model, selecting actions that are highly rated by the approximate reward function but are actually suboptimal in the real-world. Secondly, the policy may converge to local optima due to limited exploration and the absence of online feedback. To address these challenges, we introduce a new learning objective based on a function $\eta(x, a)$, called the **Maximum Action Noise Threshold (MaxANT)** function, defined as

The above is for an action that has one-dimension and is better for intuitive understanding. The following is the same formula for vectors

$$\eta(x, a) = \int_{-\delta_l}^{\delta_l} \mathcal{R}(x, a + \delta) d\delta \quad (4.4)$$

Intuitively, $\eta(x, a)$ measures the maximum positive and negative perturbation Δ that can be applied to action a without the reward falling below the reward threshold $\tau \in [r_{\min}, r_{\max}]$. This function is incorporated into the standard objective and is multiplied by the hyperparameter α to produce the modified objective

$$J(\phi) = \mathbb{E}_{x \sim p_x(\cdot)} \left[\mathcal{R}(x, \pi_\phi(x)) + \alpha \cdot \eta(x, \pi_\phi(x)) \right], \quad (4.5)$$

where the policy π_ϕ now aims to maximize not only expected reward, but also robustness to action perturbations. This encourages the selection of actions that not only have high reward but also lie in regions where nearby actions are similarly rated. As a result, the learned policy π_ϕ is less likely to exploit narrow peaks in the reward landscape and is more likely to converge to broad, high reward regions. This mitigates the risk of both reward exploitation and convergence to local optima. The standard RL objective is recovered as α approaches zero. The policy gradient $\nabla_\phi J(\phi)$ for this objective is

$$\nabla_{\phi} J(\phi) = \mathbb{E}_{x \sim p_x(\cdot)} [\nabla_{\phi} \pi_{\phi}(x) \cdot \nabla_a (r(x, a) + \alpha \cdot \eta(x, a)) |_{a=\pi_{\phi}(x)}], \quad (4.6)$$

which can be approximated by

$$\nabla_{\phi} J(\phi) = \frac{1}{B} \sum_{b=1}^B \nabla_{\phi} \pi_{\phi}(x_b) \cdot \nabla_a (r(x_b, a_b) + \alpha \cdot \eta(x_b, a_b)) |_{a_i=\pi_{\phi}(x_i)}. \quad (4.7)$$

where B represents the batch size of the sampled states. The above is an unbiased Monte Carlo estimator for the true objective. The true objective is an expectation over the possible states, but this is hard to estimate, so instead, the sampling provides an approximation of this expectation.

4.2.2 Iterative Deployment of Trained Policy

After training the policy with the modified objective described above, the learned policy π_{ϕ} is deployed in the real world to generate a dataset $\mathcal{D}_{\pi_{\phi}}$. Since the learned policy $\mathcal{D}_{\pi_{\phi}}$ is deterministic, Gaussian noise is added to the continuous actions during its deployment to evaluate its robustness. Once collected, $\mathcal{D}_{\pi_{\phi}}$ is merged with the existing dataset to form $\mathcal{D}_{i+1} = \mathcal{D}_i \cup \mathcal{D}_{\pi_{\phi}}$. The resulting dataset now contains a mixture of random exploration, expert, and on-policy data, increasing overall diversity. The new dataset is subsequently used to retrain the reward model, which in turn is used to further optimize the policy. By augmenting the dataset with on-policy samples, the reward model is refined in regions of the state-action space that are most relevant to the current policy. This helps correct suboptimal policy convergence that may arise from optimizing with an imperfect reward model though evaluation with real world interactions. The full algorithm is presented in Figure 4.1 and Algorithm 2.

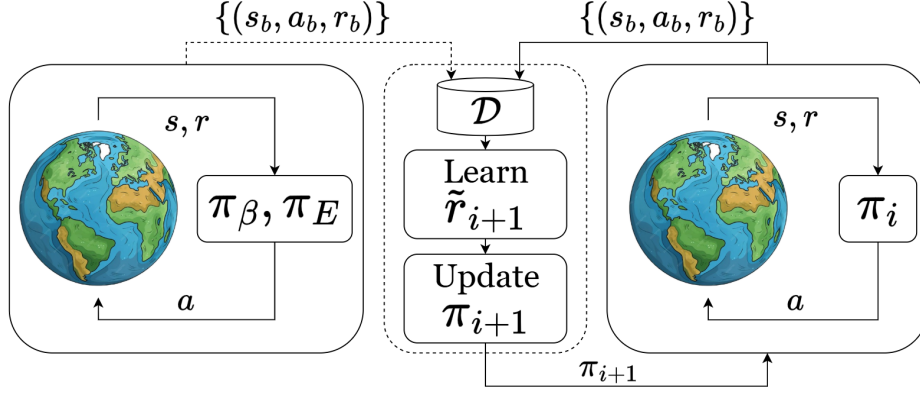


Figure 4.1: Iterative Reward Model and Policy Training. An exploration policy π_β and expert policy π_E interact with the environment to collect state-action-reward tuples, which are stored in the initial dataset $\mathcal{D}$. This dataset is used to train an approximate reward model $\tilde{r}_i$, which guides the optimization of a policy π_i . The learned policy π_i is deployed in the environment to generate on-policy data and adds it to the dataset $\mathcal{D}$. The reward model $\tilde{r}_{i+1}$ is updated with the augmented dataset, and a new policy π_{i+1} is optimized. This process repeats iteratively, progressively improving the reward model and policy.

Algorithm 2 Policy Optimization

Inputs: Number of reward networks N , Rate α

Output: Policy π_ϕ

- 1: Initialize policy network π_ϕ with random parameters ϕ
 - 2: Fill initial dataset $\mathcal{D}_0$ with data from exploration policy π_β and from expert policy π_E
 - 3: **for** $i = 0$ to I **do**
 - 4: $\tilde{r}_\theta \leftarrow \text{TrainRewardModel}(\mathcal{D}_i, N)$
 - 5: **for** $t = 1$ to T **do**
 - 6: Sample mini-batch of size B from dataset $\mathcal{D}_i$
 - 7: Update ϕ : $\nabla_\phi J(\phi) = \frac{1}{B} \sum_{b=1}^B \nabla_\phi \pi_\phi(x_b) \cdot \nabla_a (\tilde{r}_\theta(x_b, a_b) + \alpha \cdot \eta(x_b, a_b))|_{a_b=\pi_\phi(x_b)}$
 - 8: **end for**
 - 9: Execute π_{ϕ_i} and add interactions to $\mathcal{D}_{\pi_{\phi_i}}$
 - 10: Update dataset $\mathcal{D}_{i+1} = \mathcal{D}_i \cup \mathcal{D}_{\pi_{\phi_i}}$
 - 11: **end for**
 - 12: **return** π_ϕ
-

CHAPTER 5

EXPERIMENTS

To evaluate the proposed method, experiments were done on a sorting task for recyclable items with a robotic manipulator. The following experiments are the main contributions of this paper.

5.1 Experimental Setup

5.1.1 Task Description

The proposed task is to autonomously sort recyclable items within a constrained workspace. The item set consists of aluminum cans and plastic bottles with varying degrees of deformation, which introduces variability in shape and grasp difficulty. The object manipulation is performed with a KUKA LBR iiwa 3 R760 collaborative robotic arm equipped with an Ubiros Gentle Pro gripper at the end effector. The context of the workspace is captured using an overhead Oak-D S2 RGB-D camera, which provides synchronized color and depth images of the scene. These images are processed and encoded to form the input to the learned policy. The policy outputs continuous control actions for the manipulator, specifying the relative end effector placement. After executing the policy action with the corresponding motion trajectory, the robot automatically closes the gripper to secure the object, moves to the recycling bin, and opens the gripper to release any grasped object. An action is considered successful if the robot is able to securely

grasp the assigned object and transport it to the recycling bin. The experimental setup is illustrated in Figure 5.1 and the overall sorting procedure is summarized in Algorithm 3.

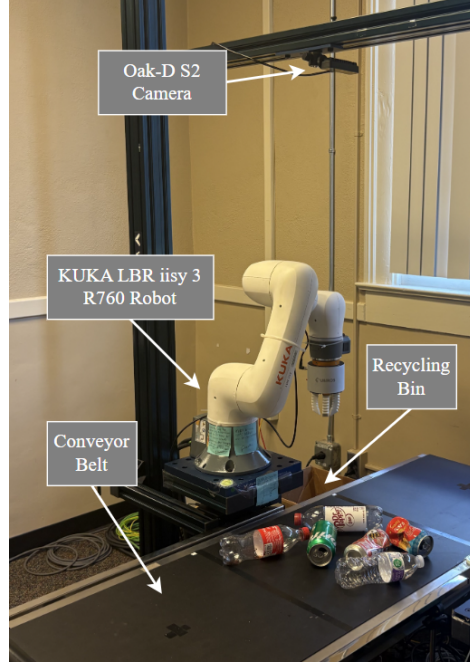


Figure 5.1: Experimental Setup. A KUKA robotic manipulator is used to grab aluminum cans and plastic bottles located on a conveyor belt. An Oak-D RGB-D camera captures images of the scene, from which the 3D position and orientation of each item is calculated. With this information, the manipulator executes grasping actions to move the recyclable items to a recycling bin

5.1.2 Context Estimation

The context of the workspace is obtained using images from an overhead RGB-D camera. When the current context is requested, the camera captures an RGB image and a corresponding depth image of the workspace. The RGB image is passed through a YOLO v11 instance segmentation model, which produces pixel-wise masks of the aluminum cans and plastic bottles present in the scene. The YOLO model was trained on a combination of publicly available datasets containing annotated images of general cans and bottle, as well as a custom dataset containing images collected directly from the workspace with manual annotation. Incorporating general and task-specific data enables the model to generalize and maintain accuracy for specific objects with the instance segmentation procedure.

Algorithm 3 Sorting

Input: Learned policy π_ϕ

```
1: while items in workspace do
2:   Camera captures RGB and Depth images of the workspace
3:   Process images to obtain context  $x$  and encoded context  $\hat{x}$ 
4:   Robot moves to "Home" position
5:   Determine action from policy:  $a \leftarrow \pi_\phi(\hat{x})$ 
6:   Robot executes action  $a$  by moving to relative end effector position
7:   Robot closes gripper to grab item
8:   Robot moves to "Bin" position
9:   Robot opens gripper to drop item
10: end while
```

After obtaining the segmented instances from the RGB image, Principal Component Analysis (PCA) (Greenacre et al., 2022) is performed on the mask of each item. PCA provides an efficient method to calculate different geometric properties from the object masks, and is first utilized to compute the centroid. The centroid represents the average position of all the pixels in the object mask, effectively providing a 2D estimate of the object’s center in the image plane. In addition to the centroid, PCA is utilized to calculate the principal axis of each mask, which is used to indicate the orientation of the each object in the image. Figure 5.2 provides a visual overview of this process, showing the initial RGB image, the extracted object masks, and the result of the PCA analysis on each mask.

The corresponding depth image is then used to recover the 3D position of the centroid from its 2D location in the image. Using the camera intrinsics and a standard pinhole model, the pixel corresponding to the centroid can be projected along a 3D ray. The depth value at the centroid’s pixel represents the distance along this ray, allowing the computation of the object’s centroid relative to the camera. This 3D position is transformed from the camera’s local coordinate frame to the robot’s global coordinate frame using ROS2 transformations.

The principal axis obtained from PCA defines the object’s yaw angle in the robot frame. However, for a complete 6D pose representation, the pitch angle must also be estimated. To compute the pitch, a two-point method is employed. Two points are selected near opposite ends of the object along the principal

axis. The 3D positions of these points are recovered from the depth image using the same strategy applied to the centroid. Let the vertical difference between the two points be Δz , and let the planar displacement in the xy -plane be Δxy . The pitch angle is then computed by $\omega = \arctan \frac{\Delta z}{\Delta xy}$, which provides a measure of the object's incline relative to the workspace plane. Due to assumed axial symmetry of the bottles and cans, the roll angle is assumed to be zero. Therefore, the full 6D pose of the each object is defined by the 3D position of the centroid and the estimated pitch and yaw angles.

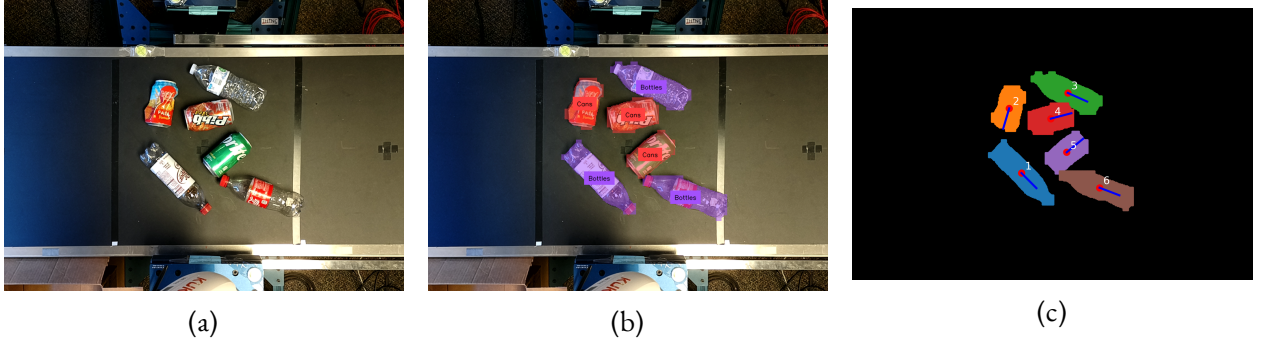


Figure 5.2: Context Estimation. Figure 5.2a shows the original RGB image. This image is passed through a YOLO v11 instance segmentation model to obtain masks of the bottles and cans, as shown in Figure 5.2b. Principal Component Analysis (PCA) is performed on the masks to determine the centroid and principal axes, shown in Figure 5.2c

5.1.3 Context Definition

Because the robot focuses on grasping one item at a time, the context representation only includes the 6D pose of the selected target. This approach greatly reduces the context space by concentrating on the object that is relevant for the current grasping action, rather than including the positions and orientations of all other objects, which is difficult when the number of items is dynamic. Therefore, the context captures the position and orientation of the active object in 3D space, along with basic contextual information about the scene. Formally the context is defined as:

$$\mathbf{x} = [x, y, z, \omega, \psi, t, n],$$

where $x, y, z \in \mathbb{R}$ represent the Euclidian coordinates of the object’s centroid, ω and ψ denote the pitch and yaw angles describing the object’s orientation, t represents the object type (e.g., can or bottle), and n denotes the number of objects currently present in the scene. Information about the object type t is necessary as the geometry of aluminum cans and plastic bottles differs. Due to the physical constraints of the workspace, the following bounds are imposed on the context variables:

$$x \in [0.20, 0.55], y \in [-0.35, 0.35], z \in [0.80, 0.95],$$

$$\omega \in [-\frac{\pi}{2}, 0], \psi \in [-\pi, \pi].$$

These bounds ensure that the context representation includes only feasible poses, preventing the robot from considering impossible positions or orientations during planning or learning.

Because the true context space is continuous, there are infinitely many possible contexts. To significantly reduce the size of the context space for the reward and policy networks, a discretized encoding is introduced. The workspace is first divided into two equal partitions along the x and y axes, producing four spatial quadrants. Each object is assigned a quadrant based on the position of its centroid, compressing all possible continuous x and y values into four discrete values. Then, the pitch angle ω range is divided into three uniform bins, and the yaw angle ψ range is divided into four uniform bins. Like the quadrant representation, the pitch and yaw bins compress the context representation into a coarse but still meaningful form. The object type t is preserved in the discretized context to maintain the distinction between different recyclable items, while the number of objects n is removed to simplify the encoding and because only a single target is considered. The resulting discretized state is compactly defined as

$$\hat{\mathbf{x}} = [q, \Omega, \Psi, t],$$

where $q \in \{1, 2, 3, 4\}$ denotes the spatial quadrant of the object, $\Omega \in \{1, 2, 3\}$ represents the discretized pitch bin, $\Psi \in \{1, 2, 3, 4\}$ denotes the discretized yaw bin, and t represents the object type. This encoding provides a simplified yet informative summary of the object and reduces the continuous context to a compact representation that is suitable for reward modeling and policy learning. Figure 5.3 shows the

discretized workspace along with an example of an item mapped to the corresponding quadrant and orientation bins. Although an encoded, discretized context $\hat{\mathbf{x}}$ is produced, the true context $\mathbf{x}$ is preserved to build an absolute pose for the gripper from the relative action (see Section 5.1.4).

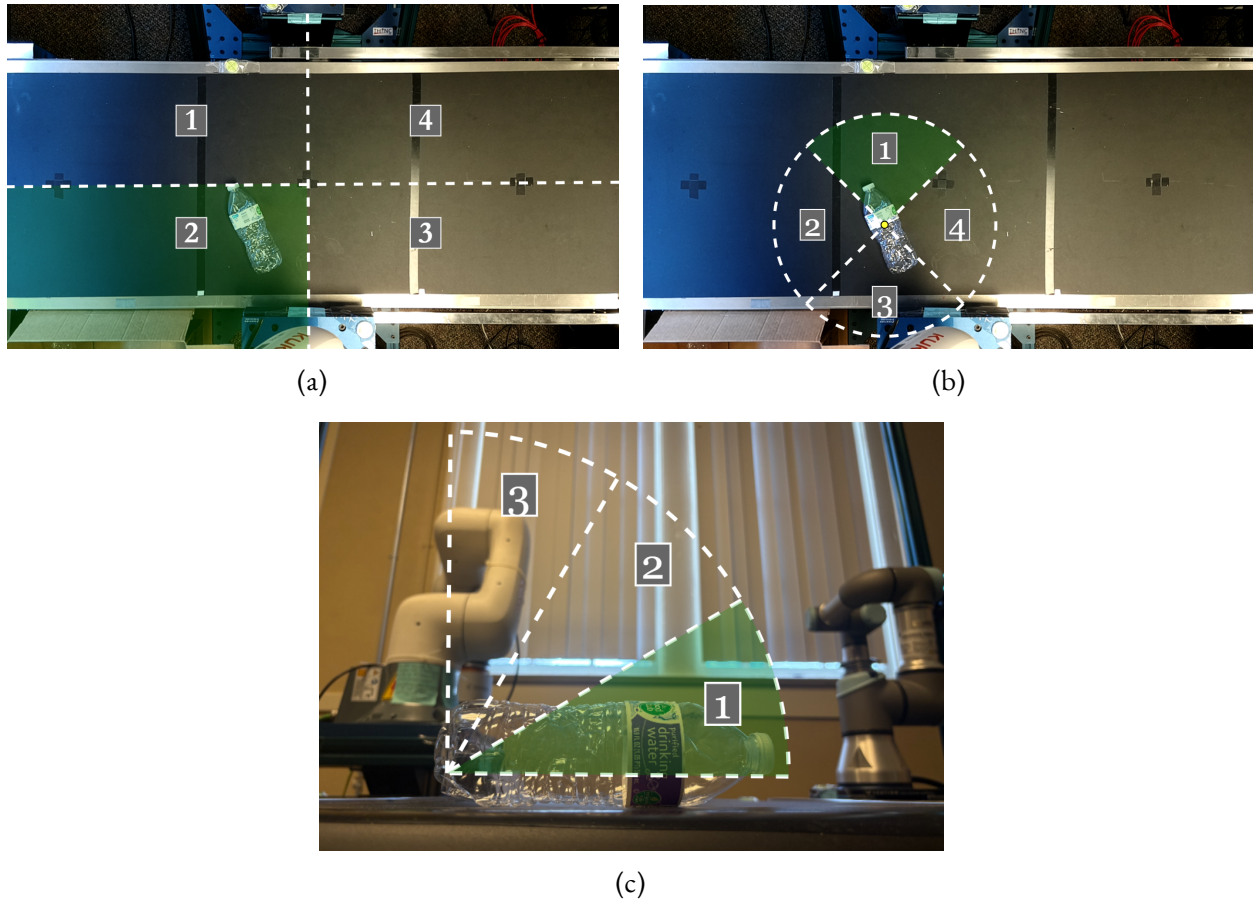


Figure 5.3: Discretized Workspace: Figure 5.3a shows the xy-plane of the workspace divided into four quadrants. In this example, the bottle is located in the second quadrant. Figure 5.3b illustrates the discretization of the possible yaw angles into four bins; the bottle falls within the first bin in this example. Similarly, the possible pitch angles are discretized into three bins, with the bottle located in the first bin, as shown in Figure 5.3c.

5.1.4 Action Definition

The actions for the sorting task define how the robotic manipulator moves prior to attempting a grasp. Rather than directly commanding joint angles or the absolute pose of the end effector in the world coordinate frame, each action specifies a relative displacement of the tool center point (TCP) or center of

the gripper with respect to the pose of the target object. In other words, the action represents a position and orientation of the gripper relative to the currently selected item. In this formulation, the sole aim of the policy is to learn gripper configurations that lead to successful grasps, without reasoning about the dynamics or planning of the robotic arm. To fully capture the 6D pose of the gripper in 3D space, the action is defined as a 6-dimensional, continuous vector:

$$\mathbf{a} = [\Delta l, \Delta w, \Delta h, \Delta \varphi, \Delta \omega, \Delta \psi],$$

where $\Delta l, \Delta w, \Delta h$ denote relative Cartesian offsets in meters along the object’s local x, y , and z axes, respectively, and $\Delta \varphi, \Delta \omega, \Delta \psi$ represent the relative rotations about the x, y , and z axes, respectively. These rotations can also be referred to as roll, pitch, and yaw.

To execute an action, the agent first computes the absolute pose with component-wise addition between the relative action $\mathbf{a}$ and the preserved, true context $\mathbf{x}$. Once the absolute pose is determined, the agent uses an inverse kinematics (Calzada-Garcia et al., 2025) solver to compute a feasible joint configuration for the target pose. If a valid solution exists, a motion planner generates a trajectory that moves the manipulator from its starting state to the desired grasping position. If no valid solution is found, the action is considered infeasible and assigned a reward of zero.

For example, the action $[0.0, 0.0, 0.0, \frac{\pi}{4}, 0.0, 0.0]$ moves the TCP to the object’s centroid with matching pitch and yaw, while introducing a relative roll offset of $\frac{\pi}{4}$ radians or 45° . Figure 5.4 illustrates the object’s local axes and provides visual clarification of how each action component maps to translations and rotations about these axes. Because all components of the action are defined in the object’s local frame, the same relative action can produce different absolute motions depending on the object’s pose in the workspace. This introduces another element of stochasticity into the task, as the same action in the same context may lead to different results.

To ensure that the robot only attempts potentially successful grasps, each component of the relative action $\mathbf{a}$ has the following bounds

$$\Delta l \in [-0.10, 0.10], \Delta w \in [-0.10, 0.10], \Delta h \in [-0.01, 0.10],$$

$$\Delta\varphi \in [-\frac{\pi}{4}, \frac{\pi}{4}], \Delta\omega \in [-\frac{\pi}{4}, \frac{\pi}{4}], \Delta\psi \in [-\frac{\pi}{4}, \frac{\pi}{4}].$$

These bounds not only prevent the agent from proposing actions that will never lead to success, but they also shrink the action space significantly. In addition to these restrictions on the relative actions, the computed absolute pose is also clamped to the same bounds defined for the context (Section 5.1.3).

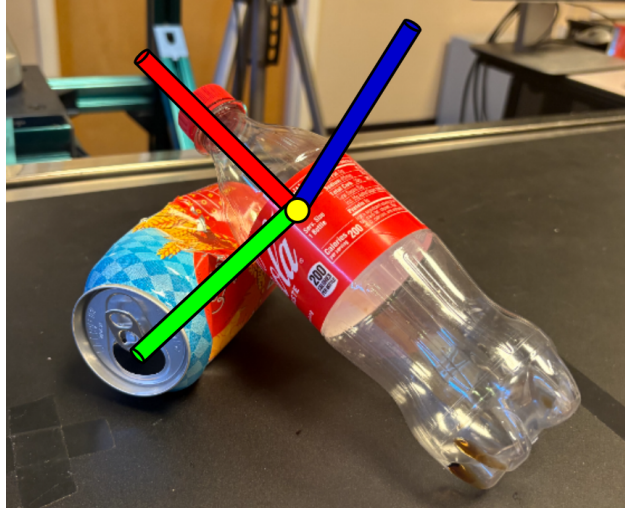


Figure 5.4: Local Coordinate Frame. The yellow dot on the bottle represents its centroid, while the red, green, and blue axes indicate the positive directions of the object’s local x , y , and z axes, respectively. An action containing a $+\Delta l$ component would produce a translation along the red axis, starting from the yellow dot.

5.1.5 Reward Definition

Rewards are assigned autonomously to each action executed in the workspace. The reward is binary: actions that result in a successful grasp receive a reward of 1, while unsuccessful actions receive a reward of 0. Action success is determined using two criteria: a reduction in the total workspace items, and the measured current load of the gripper. If the number of items in the workspace decreases by one following the action and gripper current exceeds 150 mA, the action is classified as successful and assigned a reward

of 1. Otherwise, the action is classified as a failure and assigned a reward of 0. This dual-condition check ensures that grasp attempts that fail to grab the object are not incorrectly labeled as successful.

A binary reward setup is effective for evaluating the method because multiple actions can yield successful grasps and achieve the maximum reward. However, many of these actions are suboptimal due to inherent uncertainty in the workspace and the grasping process. Rather than optimizing for any action that simply achieves success, the modified objective encourages convergence towards actions that are more optimal and robust to uncertainty. These optimal actions are often located near the center of regions with consistently high rewards. Converging to these regions improves overall performance and helps avoid policies that settle in local optima or rely on highly situational actions.

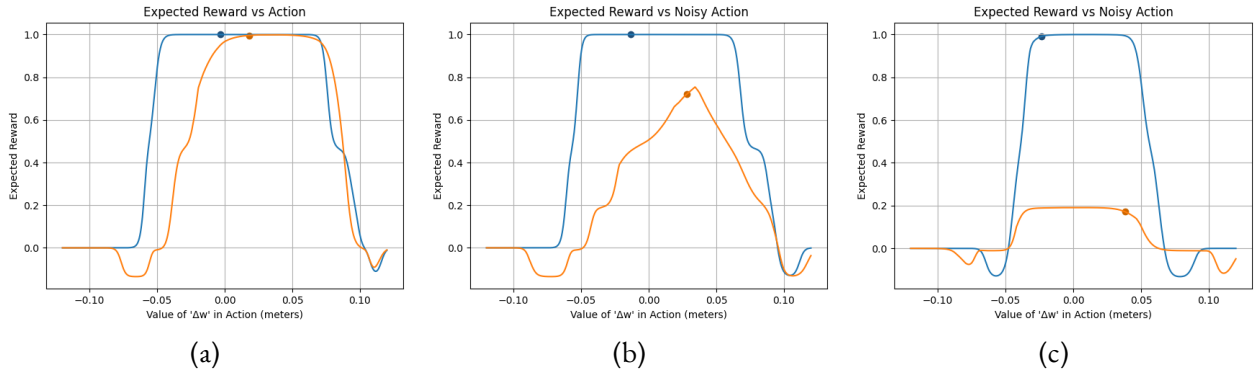


Figure 5.5: Expected Reward Comparisons: Each figure illustrates the expected reward $\mathcal{R}$ for a given state as the Δw component of the action is varied, comparing two policies trained on the same expected reward model. The blue lines correspond to the base action of the policy trained with the modified objective proposed in this work, while the orange lines show the same for the policy trained with the standard learning objective. The blue dot and orange dot indicate the respective base actions for each policy. Figure 5.5a directly compares the expected rewards of the base actions from both policies. Figures 5.5b and 5.5c show how the expected rewards change after applying the same adversarial perturbation to each action. Specifically, Figure 5.5b shows the effect of perturbing the action dimensions by 1 cm and 6° , while Figure 5.5c shows the effect of larger perturbations of 2 cm and 12° .

5.1.6 Initial Dataset

As mentioned earlier, the goal of the initial data collection phase is to produce a dataset with diverse states, actions, and rewards. To achieve this, data was collected from both an expert and a random policy to ensure a mix of successful and unsuccessful interactions. Expert data was generated by hand-guiding the KUKA

collaborative robot to a variety of feasible grasp poses for the assigned object. With this teaching process, 160 expert demonstrations were produced, corresponding to five demonstrations per state. Additionally, a random policy was implemented, sampling actions uniformly within the action bounds. Executing this policy yielded 340 additional interactions with mainly unsuccessful attempts to provide negative reward examples. Together, the expert and random data form an initial dataset of 500 interactions, containing both high quality grasps and exploration failures to produce an effective initial reward model.

CHAPTER 6

CONCLUSION

6.1 Future Work

APPENDIX A

BIBLIOGRAPHY

- Asif, U., Tang, J., & Harrer, S. (2019). Densely supervised grasp detector (dsgd). *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01), 8085–8093.
- Bagnell, J. A., Ng, A. Y., & Schneider, J. G. (2001). Solving uncertain markov decision processes.
- Bellman, R. (1957). A markovian decision process. *Journal of mathematics and mechanics*, 6(5), 679–684.
- Calzada-Garcia, A., Victores, J. G., Naranjo-Campos, F. J., & Balaguer, C. (2025). A review on inverse kinematics, control and planning for robotic manipulators with and without obstacles via deep neural networks. *Algorithms*, 18(1), 23.
- Cunningham, P., Cord, M., & Delany, S. J. (2008). Supervised learning. In *Machine learning techniques for multimedia: Case studies on organization and retrieval* (pp. 21–49). Springer.
- Detry, R., Kraft, D., Kroemer, O., Bodenhagen, L., Peters, J., Krüger, N., & Piater, J. (2011). Learning grasp affordance densities. *Paladyn*, 2(1), 1–17.
- Feinberg, V., Wan, A., Stoica, I., Jordan, M. I., Gonzalez, J. E., & Levine, S. (2018). Model-based value expansion for efficient model-free reinforcement learning. *Proceedings of the 35th International Conference on Machine Learning (ICML 2018)*.
- Fujimoto, S., Meger, D., & Precup, D. (2019). Off-policy deep reinforcement learning without exploration. *International conference on machine learning*, 2052–2062.
- Greenacre, M., Groenen, P. J., Hastie, T., d’Enza, A. I., Markos, A., & Tuzhilina, E. (2022). Principal component analysis. *Nature Reviews Methods Primers*, 2(1), 100.
- Gurney, K. (2018). *An introduction to neural networks*. CRC press.

- Haarnoja, T., Tang, H., Abbeel, P., & Levine, S. (2017). Reinforcement learning with deep energy-based policies. *International conference on machine learning*, 1352–1361.
- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *International conference on machine learning*, 1861–1870.
- Han, D., Mulyana, B., Stankovic, V., & Cheng, S. (2023). A survey on deep reinforcement learning algorithms for robotic manipulation. *Sensors*, 23(7), 3762.
- Jaiswal, A., Babu, A. R., Zadeh, M. Z., Banerjee, D., & Makedon, F. (2020). A survey on contrastive self-supervised learning. *Technologies*, 9(1), 2.
- Kumar, A., Zhou, A., Tucker, G., & Levine, S. (2020). Conservative q-learning for offline reinforcement learning. *Advances in neural information processing systems*, 33, 1179–1191.
- Langford, J., & Zhang, T. (2007). The epoch-greedy algorithm for multi-armed bandits with side information. *Advances in neural information processing systems*, 20.
- Levine, S., Pastor, P., Krizhevsky, A., Ibarz, J., & Quillen, D. (2018). Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *The International journal of robotics research*, 37(4-5), 421–436.
- Li, C., Liu, K., & Liu, S. (2025). A survey of loss functions in deep learning. *Mathematics*, 13(15), 2417.
- Li, Y. (2017). Deep reinforcement learning: An overview. *arXiv preprint arXiv:1701.07274*.
- Luo, F.-M., Xu, T., Lai, H., Chen, X.-H., Zhang, W., & Yu, Y. (2024). A survey on model-based reinforcement learning. *Science China Information Sciences*, 67(2), 121101.
- Mahler, J., Liang, J., Niyaz, S., Laskey, M., Doan, R., Liu, X., Ojea, J. A., & Goldberg, K. (2017). Dex-net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics. *arXiv preprint arXiv:1703.09312*.
- Mao, A., Mohri, M., & Zhong, Y. (2023). Cross-entropy loss functions: Theoretical analysis and applications. *International conference on Machine learning*, 23803–23828.

- Matsushima, T., Furuta, H., Matsuo, Y., Nachum, O., & Gu, S. (2020). Deployment-efficient reinforcement learning via model-based offline optimization. *arXiv preprint arXiv:2006.03647*.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015). Human-level control through deep reinforcement learning. *nature*, 518(7540), 529–533.
- Moos, J., Hansel, K., Abdulsamad, H., Stark, S., Clever, D., & Peters, J. (2022). Robust reinforcement learning: A review of foundations and recent advances. *Machine Learning and Knowledge Extraction*, 4(1), 276–315.
- Nachum, O., Dai, B., Kostrikov, I., Chow, Y., Li, L., & Schuurmans, D. (2019). Algaedice: Policy gradient from arbitrary experience. *arXiv preprint arXiv:1912.02074*.
- Nagabandi, A., Kahn, G., Fearing, R. S., & Levine, S. (2018). Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. *2018 IEEE international conference on robotics and automation (ICRA)*, 7559–7566.
- Pinto, L., & Gupta, A. (2016). Supersizing self-supervision: Learning to grasp from 50k tries and 700 robot hours. *2016 IEEE international conference on robotics and automation (ICRA)*, 3406–3413.
- Precup, D., Sutton, R. S., & Singh, S. (2000). Eligibility traces for off-policy policy evaluation.
- Prudencio, R. F., Maximo, M. R., & Colombini, E. L. (2023). A survey on offline reinforcement learning: Taxonomy, review, and open problems. *IEEE transactions on neural networks and learning systems*, 35(8), 10237–10257.
- Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014). Deterministic policy gradient algorithms. *International conference on machine learning*, 387–395.
- Slivkins, A. (2024). Introduction to multi-armed bandits. <https://arxiv.org/abs/1904.07272>
- Smith, L., Kew, J. C., Peng, X. B., Ha, S., Tan, J., & Levine, S. (2022). Legged robots that keep on learning: Fine-tuning locomotion policies in the real world. *2022 international conference on robotics and automation (ICRA)*, 1593–1599.

- Sutton, R. S. (1991). Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, 2(4), 160–163.
- Sutton, R. S., Barto, A. G., et al. (1998). *Reinforcement learning: An introduction* (Vol. 1). MIT press Cambridge.
- Tan, K. L., Esfandiari, Y., Lee, X. Y., Sarkar, S., et al. (2020). Robustifying reinforcement learning agents via action space adversarial training. *2020 American control conference (ACC)*, 3959–3964.
- Tessler, C., Efroni, Y., & Mannor, S. (2019). Action robust reinforcement learning and applications in continuous control. *International Conference on Machine Learning*, 6215–6224.
- Vecerik, M., Hester, T., Scholz, J., Wang, F., Pietquin, O., Piot, B., Heess, N., Rothörl, T., Lampe, T., & Riedmiller, M. (2017). Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards. *arXiv preprint arXiv:1707.08817*.
- Wu, P., Escontrela, A., Hafner, D., Abbeel, P., & Goldberg, K. (2023). Daydreamer: World models for physical robot learning. *Conference on robot learning*, 2226–2240.
- Xu, H., & Mannor, S. (2006). The robustness-performance tradeoff in markov decision processes. *Advances in Neural Information Processing Systems*, 19.
- Yu, T., Thomas, G., Yu, L., Ermon, S., Zou, J. Y., Levine, S., Finn, C., & Ma, T. (2020). Mopo: Model-based offline policy optimization. *Advances in neural information processing systems*, 33, 14129–14142.